

Home LLM Inference Platform

Running large language models on Kubernetes,
from the GPU up to the application

Proof that I can run LLMs in production, not just call an API.

Close the gap

28 years of infrastructure, support and PM work, but no LLM job title yet. This project supplies the evidence.

Cover both layers

Understand the model, and operate the GPU platform underneath it. Few candidates can speak to both.

Measure, not just build

Design choices are backed by monitoring data and benchmarks, and written up publicly.

About me

28 years

L3 support, professional services,
robotics engineering, marketing

10+ years

Program and project management

JP / EN

Bilingual; 3 years working in Singapore

CURRENT ROLE

American semiconductor equipment company, Japan. Founding employee of a German software company's Japan subsidiary, later acquired.
Semiconductor process optimization software.

DISTRIBUTED SYSTEMS

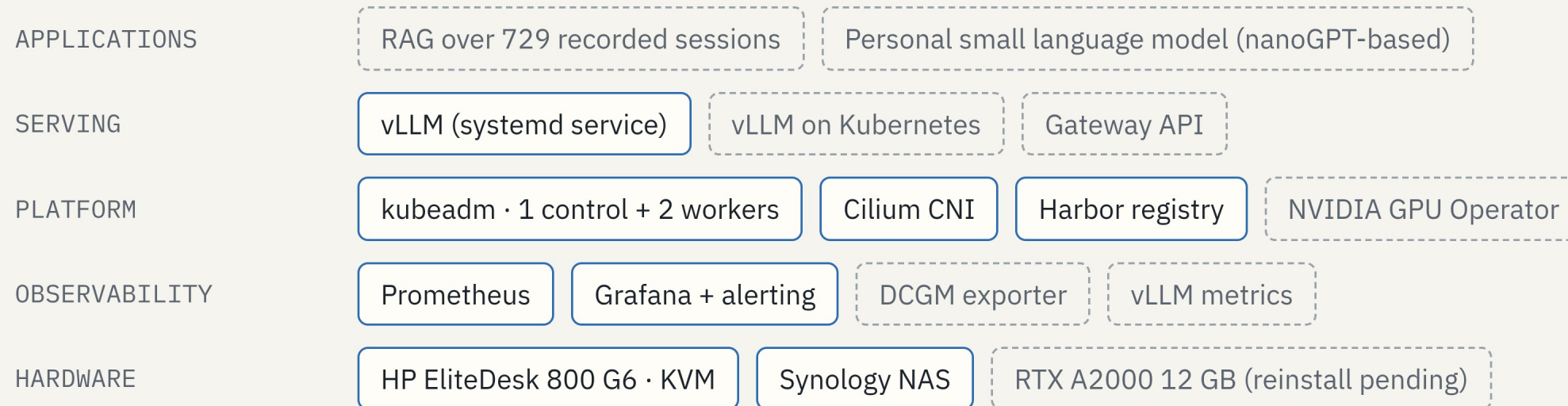
Production experience with Apache Ignite and Microsoft Orleans

CERTIFICATIONS

NVIDIA NCA-AIIO · CNCF KCNA · CCNA · SCJP

Architecture

Solid = running today · Dashed = planned



Where it stands today

Component	Status	Notes
kubeadm cluster	Running	1 control plane + 2 workers on KVM VMs
Cilium CNI	Running	Enforces standard NetworkPolicy
Harbor registry	Running	Self-hosted image source for the cluster
vLLM (Phi-3-mini-4k-instruct)	Running	systemd service; API tested from a remote client
Prometheus + Grafana	Running	Host metrics and custom memory alerting
GPU worker + GPU Operator	Next	Waiting on the GPU power fix
vLLM on Kubernetes	Planned	Deployment, Harbor images, Gateway API
GPU and LLM dashboards	Planned	DCGM exporter and vLLM metrics

Roadmap in five phases

PHASE 1

GPU in the cluster

Reinstall the RTX A2000, pass it to a worker VM, deploy the NVIDIA GPU Operator.

PHASE 2

LLM serving on Kubernetes

Move vLLM to a Deployment, pull images from Harbor, expose it through Gateway API.

PHASE 3

Observability

DCGM for GPU health, vLLM metrics for latency and throughput, Grafana dashboards.

PHASE 4

Benchmarks

Compare quantization, concurrency and context length on a 12 GB GPU. Publish results.

PHASE 5

Applications

Run RAG over my recorded sessions and a personal small language model on top.

Phases 1–3: about 1–2 months · Phase 4: about 1 month · Phase 5: ongoing

Questions the benchmarks will answer

Question	Variable	Metrics	Result
How many concurrent users fit on 12 GB?	Concurrent requests	Tokens/s, p95 latency	[TBD]
What does quantization trade away?	FP16 vs 4-bit	Tokens/s, VRAM, quality	[TBD]
How long can the context get?	Max model length	VRAM, time to first token	[TBD]
Which model size fits best?	Small vs mid-size models	Tokens/s, VRAM	[TBD]

Every result is published with its method, so anyone can reproduce it.

What broke, and how I fixed it

SYMPTOM

Long-running scripts killed whenever the remote desktop disconnected

ROOT CAUSE

systemd ended the user session (Linger=no)

FIX

loginctl enable-linger

SYMPTOM

Hardware video acceleration not working on a Haswell machine

ROOT CAUSE

Wrong VA-API driver for that GPU generation

FIX

i965 driver instead of iHD

SYMPTOM

Power error on the host after adding the GPU

SUSPECTED CAUSE

Stock 260 W power supply

STATUS: OPEN

GPU removed safely; fix in progress

What this demonstrates

Skill	Evidence
Kubernetes administration	kubeadm cluster with Cilium and Harbor; CNCF KCNA
GPU infrastructure	GPU passthrough on KVM, CUDA; NVIDIA NCA-AIIO
LLM serving	vLLM as a managed service with an OpenAI-compatible API
Observability	Prometheus, Grafana and custom alerting in daily use
Troubleshooting	L3 support background; documented break-fix log
Communication	Write-ups published in Japanese and English

Next steps

CERTIFY

NVIDIA NCP-GENL for LLM depth. NCP-AIO added if targeting platform roles.

PUBLISH

Public GitHub repo, plus write-ups on Zenn (Japanese) and LinkedIn (English).

BUILD ON IT

Applications on the platform: RAG over recorded sessions and a personal SLM.